

Deployment

Part of the [React curriculum](#). This document covers static hosting, containerized deployment, CI/CD for a frontend build, service-worker caching and edge-gated routes, and the special case where the build itself is the release.

1. The build produces static files

Unlike the Python API ([Python API Backend, Deployment](#)), a React app built with a tool like Vite compiles down to a set of **static files** — HTML, JavaScript, CSS, and assets — with no server-side runtime of its own required to serve them:

```
npm run build
# produces a dist/ directory of static files
```

This is a meaningfully simpler deployment shape than a backend service: there's no process to keep running, no server-side crash to recover from — just files that need to be served over HTTP, which almost any static file host or CDN can do.

2. Static hosting platforms

Option	What it is
Netlify / Vercel	Purpose-built static-site/frontend hosting — connect a Git repository, and the platform builds and deploys on every push automatically, including preview deployments per pull request
Cloud object storage + CDN (S3 + CloudFront, Google Cloud Storage + Cloud CDN)	Upload the built files to object storage, serve them through a CDN in front of it for caching and global distribution
A plain web server (nginx, serving files directly)	Simplest possible option if you're already running your own infrastructure

For most projects, a purpose-built platform (Netlify/Vercel) is the least-effort option and handles CDN distribution, HTTPS, and build automation for you; object storage + CDN is the more manual, more configurable alternative common in larger cloud-native deployments.

3. Client-side routing needs a server-side fallback rule

Since routing (§1 of [Routing & API Integration](#)) happens entirely in the browser via JavaScript, the *server* needs to be configured to serve the app's single `index.html` for **any** path the app defines a route for — not just the literal root path. Without this, directly loading (or refreshing) a URL like `/detail/123` would 404 at the server level, since

no actual file named `detail/123` exists; only client-side JavaScript (which hasn't loaded yet, because the server never returned `index.html` in the first place) knows how to render that route.

```
location / {  
    try_files $uri /index.html;  
}
```

Most static hosting platforms (Netlify, Vercel) handle this automatically for a detected SPA; configuring it yourself (as above) is necessary if you're serving the build from a plain nginx server or a generic object-storage + CDN setup.

4. Containerized deployment

If you'd rather deploy the built static files as part of a container (e.g., to keep it consistent with how a colocated backend is deployed), a common pattern is a two-stage Dockerfile: build the app in one stage, then copy only the built static output into a minimal nginx image:

```
FROM node:20 AS build  
WORKDIR /app  
COPY package*.json ./  
RUN npm ci  
COPY . .  
RUN npm run build  
  
FROM nginx:alpine  
COPY --from=build /app/dist /usr/share/nginx/html  
COPY nginx.conf /etc/nginx/conf.d/default.conf  
EXPOSE 80
```

This **multi-stage build** keeps the final image small — it contains only nginx and the built static files, not the entire Node.js toolchain and `node_modules` needed to produce them.

5. Environment configuration at build time vs. deploy time

Recall from [Routing & API Integration, §8.2](#) that Vite bakes environment variables in at **build time**, not read fresh at container/process start the way a backend service typically does. This means "one build, configured per environment via env vars at deploy time" (the ideal from [the architecture overview, §2.7](#)) doesn't apply quite as directly to a frontend build as it does to a backend — if a value genuinely needs to differ between environments, you either produce a separate build per environment, or read the value at runtime instead of build time (e.g., fetching a small runtime config file the app loads on startup, rather than baking the value into the JavaScript bundle itself).

6. CI/CD

A typical pipeline for a frontend build:

1. **On every push:** run the test suite ([Testing](#)), a linter, and a type check (`tsc --noEmit`).
2. **On merge to main** (or a release trigger): run `npm run build`, producing the static output.
3. **Deploy:** upload the built files to the hosting platform/CDN, or build and push a container image if using the containerized approach (§4).

As with the backend ([Python API Backend, Deployment, §6](#)), the key property to preserve is that nothing reaches production without first passing the automated test suite and type check.

7. Progressive web apps and edge-gated routes

A service worker (generated by a plugin such as Workbox) makes the app installable and fast on repeat visits, and introduces a cache you must now reason about on every release.

- **Pick a strategy per kind of request.** Built, content-hashed JS/CSS/icons: precache and serve from the cache. API calls: **network-first with a short timeout** (a few seconds) that falls back to a cached copy only when the network is down — for live data, freshness matters more than offline access. Choose `registerType: 'autoUpdate'` so a new worker takes over without asking.
- **A navigation fallback needs a deny-list.** The service worker answers a top-level navigation to any unknown path with the cached app shell (that is how client-side routing works offline). Two classes of path must be **excluded** from that fallback, or the worker will answer them with the SPA instead of letting the request reach the server:
 1. any path with a **file extension** (a direct link to a downloadable `.pptx / .pdf` should download, not render `index.html`);
 2. any path the **backend or edge owns** — API prefixes, and especially routes gated by an identity-provider login. A gated page served from the cached shell never touches the edge, so an expired session is *masked*: the user sees a stale page whose API calls then fail. Build the deny-list in one tested function and extend it for every new gated route. The pattern in the generated worker must be a regex literal, not a function closing over a build-time variable — the worker is serialised to a standalone file where that variable doesn't exist.
- **The worker script itself must not be cached.** Serve `sw.js` with `no-cache` (or `must-revalidate`), including from any CDN or edge rule in front of it; a worker cached for a day delays every release by a day.
- **Verifying a deploy needs a clean browser state.** A browser profile that has already visited keeps its registered worker, so "open a new tab" is not enough to see a new release. Unregister the worker (DevTools → Application) and reload with a cache-busting query, or use a fresh profile; in an automated check, assert the content-hashed asset filename the page loads.
- **Gated fetches need the identity provider in the page's CSP.** A page's own `fetch()` to an endpoint behind an identity-provider login follows a redirect to the provider's domain; unless that domain is allowed in the content-security `connect-src`, the redirect fails silently and the call looks like a network error.

8. When the build *is* the release

If the production server simply serves the build directory from disk (a static preview server, for instance), then **running the build command is itself the point of no return**: the new files are live the moment they are written, before any restart or confirmation. Consequences:

- Build only when you are ready to ship, and treat the build as the deploy step in the checklist, not as a preparatory one. Keep the previous output (or be able to rebuild a known commit quickly) as the rollback.
- If a build-time variable such as the base path is baked into the output, the build must be run **with the production value**; restarting the server never fixes a wrong one.
- If a shared design system is consumed from **source** by path alias (no published package), update that checkout *first* and install its dependencies if they changed, then pull and build the app — the app's build reads the dependency's current source, so building against a stale checkout fails or ships the wrong components.
- Editing a service definition's environment usually needs the service manager to *unload and reload* the job, not just restart it; a restart re-runs the already-loaded definition and silently keeps the old environment.
- Confirm with evidence, not feel: the content-hashed bundle filename the live page requests is the reliable proof that the fresh build, rather than a cached or stale one, is serving.

9. Core tools summary

Tool	Used for
Vite's <code>build</code> command	producing the deployable static output
Netlify / Vercel	zero-infrastructure static hosting with automatic Git-based deploys
nginx	serving the built files yourself, with the SPA fallback rule (§3)
Docker (multi-stage build)	packaging the build alongside a colocated backend's own deployment
vite-plugin-pwa (Workbox)	service worker generation, precache and runtime caching rules (§7)

See also

- [React curriculum index](#)
- [Testing](#) — verifying the app before it's deployed
- [Architecture Overview](#), §11 for how this fits alongside the API's own deployment